

1.what is javascript?

JavaScript is a high-level, dynamic, multi-paradigm programming language used to create dynamic and interactive web pages and applications. It allows developers to handle user interactions, manipulate web content, communicate with servers, and build complete applications. JavaScript can run on both the client side (browser) and server side, using environments such as Node.js.

2.Client-side vs Server-side JavaScript

Client-side JavaScript is JavaScript that runs in the user's web browser. It is mainly used to make web pages interactive, such as handling button clicks, validating forms, changing HTML/CSS, creating animations, and updating content without reloading the page.

Server-side JavaScript is JavaScript that runs on the server, usually using an environment such as Node.js. It is mainly used for backend tasks such as handling requests, creating APIs, communicating with databases, authentication, and processing application logic.

Client-side	Server-side
Runs in the browser	Runs on the server
Handles UI and interaction	Handles backend logic
Can access the DOM	Cannot directly access the browser DOM
Example: button click	Example: database operation
Uses browser JS engines	Commonly uses Node.js/V8

3. Javascript Engine

A JavaScript Engine is a software program that reads, interprets, compiles, and executes JavaScript code so that the computer can perform the instructions written in JavaScript.

In simple words:

JavaScript Engine is the component that converts JavaScript code into instructions that the computer can execute.

Examples of JavaScript Engines:

Engine	Used in
V8	Google Chrome, Node.js
SpiderMonkey	Mozilla Firefox
JavaScriptCore	Safari
Chakra	Older Microsoft Edge

4. ECMA Script

ECMAScript is a standardized specification that defines how scripting languages like JavaScript should work. It specifies the rules, syntax, features, data types, objects, and behavior that a JavaScript implementation should follow.

In simple terms:

ECMAScript is the standard, while JavaScript is a programming language that follows that standard.

It defines things like:

- Variables and data types
- Functions
- Loops and conditions
- Objects and classes
- Syntax and language features
- New features added to JavaScript

5. JavaScript Execution

JavaScript execution simply means running JavaScript code and carrying out the instructions written in it.

For example:

```
let a = 10;
```

```
let b = 20;
```

```
console.log(a + b);
```

When this code runs, the JavaScript engine processes it and produces:

30

6.Variable

A variable is like a container used to store data that we can use later in our program.

JavaScript provides three keywords for declaring variables: var, let, and const. They differ mainly in their redeclaration, reassignment, and scope.

var

```
var age;
```

```
age = 25;
```

```
console.log(age);
```

// var is redeclarable and reassignable and has function scope.

let

```
let name;
```

```
name = "Ups";
```

```
console.log(name);
```

// let is not redeclarable, but is reassignable and has block scope.

// let name = "JavaScript"; // Redeclaration not allowed

// name = "Java"; // Reassignment allowed

const

const country = "Nepal";

console.log(country);

//const is not redeclarable and not reassignable and has block scope.

// const country = "India"; // Redeclaration not allowed

// country = "India"; // Reassignment not allowed

7. Scope

Scope means the area of a program where a variable can be accessed or used.

JavaScript mainly has:

- Global Scope → accessible throughout the program.
- Function Scope → accessible within a function. var follows this scope.
- Block Scope → accessible only inside { }. let and const follow this scope.

8. Hoisting

Hoisting means JavaScript knows about declarations before it executes the code.

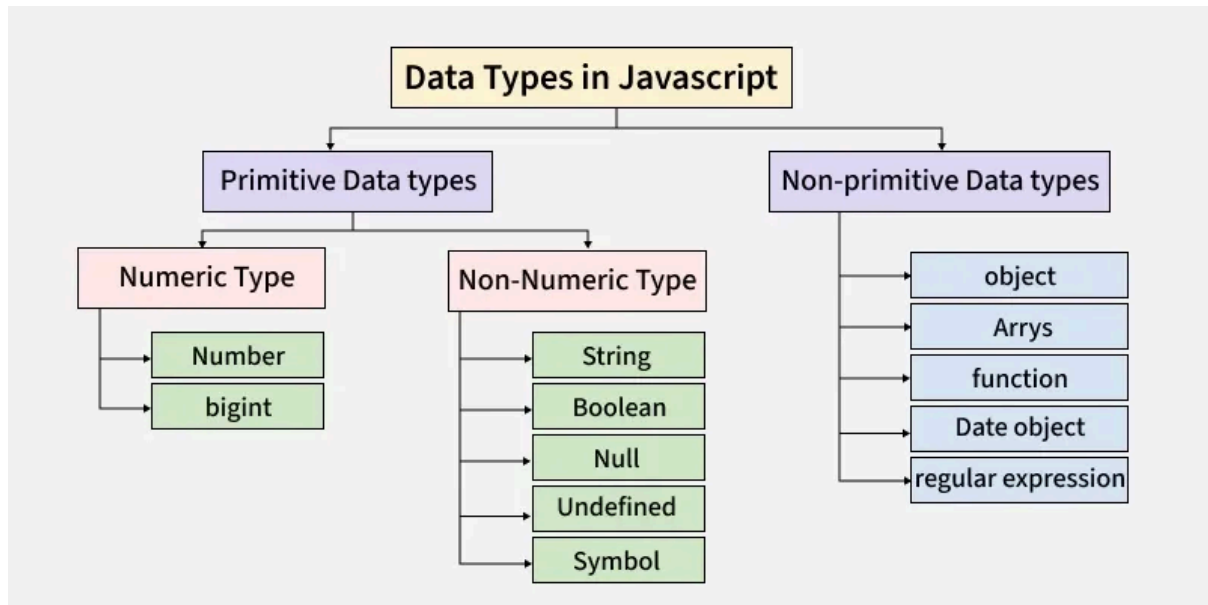
var age;

console.log(age); // undefined

age = 25;

Here its look like error but gives undefined.

9. Data Types



Primitive Data Types

Primitive types store a single value.

There are 7 primitive data types:

Data Type	Example	Meaning
String	"Hello"	Text
Number	25, 10.5	Numbers
BigInt	123456789n	Very large integers
Boolean	true, false	True or false
Undefined	let x;	Value not assigned
Null	null	Intentionally empty value
Symbol	Symbol("id")	Unique value

Non-Primitive Data Types

Non-primitive data types are used to store collections of data or more complex values, rather than a single simple value.

The main non-primitive data types shown in the figure are:

1. Object

An object stores data in the form of key-value pairs.

```
let student = {  
  name: "Upskill",  
  age: 25  
};
```

Here, name and age are properties of the object.

2. Array

An array is used to store multiple values in an ordered collection.

```
let subjects = ["JavaScript", "HTML", "CSS"];
```

Each value has an index, starting from 0.

```
console.log(subjects[0]); // JavaScript
```

3. Function

A function is a reusable block of code that performs a specific task.

```
function greet() {  
  console.log("Hello!");  
}
```

```
greet();  
\
```

4. Date Object

The Date object is used to work with dates and times.

```
let today = new Date();
```

```
console.log(today);
```

It can be used to get the current date, time, year, month, etc.

5. Regular Expression

A regular expression (RegExp) is used to search, match, or validate patterns in text.

```
let pattern = /hello/;
```

```
console.log(pattern.test("hello world")); // true
```

It is commonly used for things like text searching and form validation.

10. Operator

Operators are symbols or keywords used to perform operations on values and variables.

For example:

```
let a = 10;
```

```
let b = 5;
```

```
console.log(a + b); // 15
```

Here, + is an operator used to add two values.

JavaScript has several types of operators, including Arithmetic, Assignment, Comparison, Logical, and Ternary operators.

1. Arithmetic Operators

Arithmetic operators are used to perform mathematical calculations.

Operator	Meaning	Example	Result
+	Addition	10 + 5	15
-	Subtraction	10 - 5	5
*	Multiplication	10 * 5	50
/	Division	10 / 5	2
%	Modulus (remainder)	10 % 3	1

**	Exponentiation	2 ** 3	8
++	Increment	x++	Increases by 1
--	Decrement	x--	Decreases by 1

Example:

```
let a = 10;
```

```
let b = 3;
```

```
console.log(a + b); // 13
```

```
console.log(a % b); // 1
```

2. Assignment Operators

Assignment operators are used to assign or update values in variables.

Operator	Example	Equivalent to
=	x = 10	Assign 10
+=	x += 5	x = x + 5
-=	x -= 5	x = x - 5
*=	x *= 5	x = x * 5
/=	x /= 5	x = x / 5
%=	x %= 5	x = x % 5

Example:

```
let x = 10;
```

```
x += 5;
```

```
console.log(x); // 15
```

3. Comparison Operators

Comparison operators are used to compare two values. They return either true or false.

Operator	Meaning	Example
==	Equal value	5 == "5" → true
===	Equal value and type	5 === "5" → false
!=	Not equal	5 != 3 → true
!==	Not equal value or type	5 !== "5" → true
>	Greater than	10 > 5 → true
<	Less than	10 < 5 → false
>=	Greater than or equal	10 >= 10 → true
<=	Less than or equal	5 <= 10 → true

4. Logical Operators

Logical operators are used to combine or reverse conditions.

Operator	Name	Example
&&	AND	true && true → true
,		,
!	NOT	!true → false

Example:

```
let age = 25;
```

```
let hasLicense = true;
```

```
console.log(age >= 18 && hasLicense);
```

```
// true
```

Here, both conditions must be true for && to return true.

5. Ternary Operator

The ternary operator is a short way of writing a simple if...else condition.

Syntax

```
condition ? valueIfTrue : valueIfFalse;
```

Example:

```
let age = 20;
```

```
let result = age >= 18 ? "Adult" : "Minor";
```

```
console.log(result);
```

Output:

Adult

The same logic using if...else would be:

```
let result;
```

```
if (age >= 18) {  
    result = "Adult";  
} else {  
    result = "Minor";  
}
```